

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250284564

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

COMBEE; Benjamin et al.

OPTIMIZED RENDERING OF A USER INTERFACE

Abstract

Provided herein are systems, apparatuses, articles of manufacture, methods and/or computer program product embodiments, and/or combinations and sub-combinations thereof, for optimizing the rendering of a user interface (UI). For example, a first set of drawing instructions configured to render a first UI element for a graphical UI (GUI) is obtained. A determination is made that a representation of the first UI element is stored in a memory. In response to determining that the representation of the first UI element is stored in the memory, execution of the first set of drawing instructions is bypassed. The representation of the first UI element is obtained from the memory based on bypassing execution of the first set of drawing instructions. The obtained representation of the first UI element is rendered for the GUI.

Inventors: COMBEE; Benjamin (Austin, TX), KATCHIK; Nikita (Barnet, GB), CHOW; Michael (San Francisco, CA)

Applicant: ROKU, INC. (San Jose, CA)

Family ID: 1000008619743

Assignee: ROKU, INC. (San Jose, CA)

Appl. No.: 19/214504

Filed: May 21, 2025

Related U.S. Application Data

parent US continuation 17825164 20220526 parent-grant-document US 12131202 child US 18830189

parent US continuation-in-part 18830189 20240910 PENDING child US 19214504

us-provisional-application US 63194749 20210528

Publication Classification

Int. Cl.: G06F9/54 (20060101); **H04N21/81** (20110101)

U.S. Cl.:

CPC G06F9/541 (20130101); **G06F9/542** (20130101); **G06F9/547** (20130101); **H04N21/8173** (20130101)

Background/Summary

RELATED APPLICATION [0001] This application is a continuation-in-part of U.S. patent application Ser. No. 18/830,189, filed Sep. 10, 2024, which is a continuation of U.S. patent application Ser. No. 17/825,164, filed May 26, 2022 (now U.S. Pat. No. 12,131,202 and issued on Oct. 29, 2024), which claims priority to U.S. Provisional Application No. 63/194,749, filed on May 28, 2021. The entirety of each of these applications is incorporated by reference herein.

BACKGROUND

Field

[0002] This disclosure is generally directed to applications on media devices, and more particularly to cloud computation for applications on media devices.

Background

[0003] Media systems can include media devices, such as a streaming media device, a DVD or BLU-RAY device, a television (TV), an audio/video playback device, a cable box, and/or a digital video recording device, to name just a few examples. Compared to a generic computing device, a media device may have more functions designed for playing media content such as video and audio content, but limited functions for other kinds of computing. Hence, applications operating on a media device may run into various limitations. Improved solutions for running applications on media devices are therefore needed.

SUMMARY

[0004] Provided herein are systems, apparatuses, articles of manufacture, methods and/or computer program product embodiments, and/or combinations and sub-combinations thereof, for optimizing the rendering of a user interface (UI). For example, a first set of drawing instructions configured to render a first UI element for a graphical UI (GUI) is obtained. A determination is made that a representation of the first UI element is stored in a memory. In response to determining that the representation of the first UI element is stored in the memory, execution of the first set of drawing instructions is bypassed. The representation of the first UI element is obtained from the memory based on bypassing execution of the first set of drawing instructions. The obtained representation of the first UI element is rendered for the GUI.

[0005] Also provided herein are systems, apparatuses, articles of manufacture, methods and/or computer program product embodiments, and/or combinations and sub-combinations thereof, for developing and operating a user application or an application on a media device or a user device facilitated by a cloud computing system. A user application can include a user rendered context operating on a user device and maintaining a current render tree, and a user programmer context operating on a cloud computing system. The user renderer context can be executed on the user device and is responsible for providing input or output functions for the user application. The user programmer context is a part of the user application that is executed in the cloud computing system and is responsible for operations related to business logic, application flow, data model, or responses to user navigations. In some embodiments, the user application can be written in one language for both the user rendered context and the user programmer context.

[0006] The user rendered context of the user application can include an application user interface

presented within a user-renderer frame on a display of the user device. In some embodiments, the user-renderer frame is contained within a root window of a GUI for a web browser installed on the user device. The user-rendered context can generate drawing instructions to an abstracted two-dimensional drawing surface without running HyperText Markup Language (HTML) in a browser. The user-rendered context can include one or more small components, and a small component is responsible for low-level behavior including key responses, animations, or graphical appearance. [0007] The user-rendered context can start operation after the user-programmer context is initialized on the cloud computing system. The user-rendered context can receive an input; and send, responsive to receiving the input, an event notification to the user-programmer context of the user application. The user-rendered context can be initialized with an initial render tree provided from the user-programmer context. The user-rendered context can further receive, from the user-programmer context, a difference tree to update the current render tree, and update the current render tree based on the difference tree. In some embodiments, the difference tree received from the user-programmer context operated in the cloud computing system is represented by JavaScript Object Notation (JSON).

Description

BRIEF DESCRIPTION OF THE FIGURES

[0008] The accompanying drawings are incorporated herein and form a part of the specification.

[0009] FIG. 1 illustrates a block diagram of a multimedia environment, according to some embodiments.

[0010] FIG. 2 illustrates a block diagram of a streaming media device, according to some embodiments.

[0011] FIG. 3 illustrates an example clouding computing system to perform computations for a user device, according to some embodiments.

[0012] FIG. 4 illustrates an example software process for developing an application operating on a user device and a cloud computing system, according to some embodiments.

[0013] FIGS. 5-6 illustrate example architectural overviews of applications operating on a user device and a cloud computing system, according to some embodiments.

[0014] FIGS. 7-8 illustrate example sequence diagram of operations performed by an application operating on a user device and a cloud computing system, according to some embodiments.

[0015] FIG. 9 is a block diagram of a media device configured to optimize the rendering of a GUI, according to some embodiments.

[0016] FIG. 10 is a flowchart for a method for rendering a cached representation of a UI element for a GUI, according to some embodiments.

[0017] FIG. 11 is a flowchart for a method for determining whether a representation of a first UI element is stored in memory, according to some embodiments.

[0018] FIG. 12 is a flowchart for a method for determining whether a representation of a first UI element is stored in memory, according to some embodiments.

[0019] FIG. 13 is a flowchart for a method for executing a set of drawings instructions when a representation of a UI element corresponding to the set of drawing instructions is not cached, according to some embodiments.

[0020] FIG. 14 is a flowchart for a method for determining whether a representation of a second UI element is stored in memory, according to some embodiments.

[0021] FIG. 15 is a flowchart for a method for rendering a representation of a second UI element, according to some embodiments.

[0022] FIG. 16 illustrates an example computer system useful for implementing various embodiments.

[0023] In the drawings, like reference numbers generally indicate identical or similar elements. Additionally, generally, the left-most digit(s) of a reference number identifies the drawing in which the reference number first appears.

DETAILED DESCRIPTION

[0024] A media device, such as a streaming media device, a DVD or BLU-RAY device, an audio/video playback device, a cable box, a television (TV), and/or a digital video recording device, may have more functions designed for playing media content such as video and audio content, but limited functions for other kinds of computing. For example, the input device to a media device may be a remote control instead of a fully functional keyboard. And an application operating on a media device can have different limitations than applications operating on a generic computing device. An application can be referred to as a user application as well. For example, a media device may not support a normal web browser due to processing and storage demands. A media device may be referred to as a user device, and an application can be referred to as an app.

[0025] Embodiments herein present a cloud-based execution environment, providing a new way of execution for applications or channels. An application can be separated into two parts, a user renderer context operating on a media device, and a user program context operating on a cloud computing system. In some embodiments, the user program context can be referred to as a user programmer context as well. Functions in an application can be separated into two parts, where some functions are implemented in the user renderer context in small components, and where some other functions are implemented in the user program context in big components. The user renderer context can be executed on the media device and is responsible for the user input/output, such as graphical user interface (GUI), accepting inputs such as keystrokes, drawing to the screen, and receiving and reacting to user inputs. The user program context is the part of the application that can be executed in the cloud computing system and is responsible for the business logic, application flow, data model and responses to user navigation. The user renderer context can receive an update from the user program context to generate new user interface (UI) from a previous UI. The cloud computing system can maintain a container that stores multiple trees used to render UIs for multiple user renderer contexts on different user devices.

[0026] Embodiments herein also present a cloud-based development environment, providing an improved way of developing apps/channels. Currently, some development environments use proprietary languages/programs, and apps under the development have to run through the operating system in a media device (OS). Embodiments herein can be based on existing standards like Javascript rather than proprietary models, and can execute in the cloud rather than on the device, and can execute in a web browser for development, testing, and debugging purposes.

[0027] Embodiments herein can have numerous advantages, including but not limited to speeding up development, keeping devices affordable, increasing ease of use by developers familiar with standard languages, improving availability of debugging tests and other development tools that work on standards rather than proprietary languages/programs.

[0028] Various embodiments of this disclosure may be implemented using and/or may be part of a multimedia environment **102** shown in FIG. **1**. It is noted, however, that multimedia environment **102** is provided solely for illustrative purposes, and is not limiting. Embodiments of this disclosure may be implemented using and/or may be part of environments different from and/or in addition to multimedia environment **102**, as will be appreciated by persons skilled in the relevant art(s) based on the teachings contained herein. An example of multimedia environment **102** shall now be described.

Multimedia Environment

[0029] FIG. **1** illustrates a block diagram of multimedia environment **102**, according to some embodiments. Multimedia environment **102** illustrates an example environment, architecture, ecosystem, etc., in which various embodiments of this disclosure may be implemented. However, multimedia environment **102** is provided solely for illustrative purposes, and is not limiting.

Embodiments of this disclosure may be implemented and/or used in environments different from and/or in addition to multimedia environment **102** of FIG. **1**, as will be appreciated by persons skilled in the relevant art(s) based on the teachings contained herein.

[0030] In a non-limiting example, multimedia environment **102** may be directed to streaming media. However, this disclosure is applicable to any type of media (instead of or in addition to streaming media), as well as any mechanism, means, protocol, method and/or process for distributing media.

[0031] Multimedia environment **102** may include one or more media systems **104**. Media system **104** could represent a family room, a kitchen, a backyard, a home theater, a school classroom, a library, a car, a boat, a bus, a plane, a movie theater, a stadium, an auditorium, a park, a bar, a restaurant, or any other location or space where it is desired to receive and play streaming content. User(s) **132** may operate with media system **104** to select and consume content.

[0032] Each media system **104** may include one or more media devices **106** each coupled to one or more display devices **108**. It is noted that terms such as “coupled,” “connected to,” “attached,” “linked,” “combined” and similar terms may refer to physical, electrical, magnetic, logical, etc., connections, unless otherwise specified herein.

[0033] Media device **106** may be a streaming media device, a streaming set-top box (STB), cable and satellite STB, a DVD or BLU-RAY device, an audio/video playback device, cable box, and/or a digital video recording device, to name just a few examples. Display device **108** may be a monitor, a television (TV), a computer, a computer monitor, a smart phone, a tablet, a wearable (such as a watch or glasses), an appliance, an internet of things (IoT) device, and/or a projector, to name just a few examples. Both media device **106** and display devices **108** can be referred to as a user device. In some embodiments, media device **106** can be a part of, integrated with, attached to, operatively coupled to, and/or connected to its respective display device **108**.

[0034] Each media device **106** may be configured to communicate with network **118** via a communication device **114**. Communication device **114** may include, for example, a cable modem or satellite TV transceiver. Media device **106** may communicate with communication device **114** over a link **116**, wherein link **116** may include wireless (such as WiFi) and/or wired connections.

[0035] In various embodiments, network **118** can include, without limitation, wired and/or wireless intranet, extranet, Internet, cellular, Bluetooth, infrared, and/or any other short range, long range, local, regional, global communications mechanism, means, approach, protocol and/or network, as well as any combination(s) thereof.

[0036] Media system **104** may include a remote control **110**. Remote control **110** can be any component, part, apparatus and/or method for controlling media device **106**, display device **108**, such as a remote control, a tablet, laptop computer, smartphone, wearable, on-screen controls, integrated control buttons, audio controls, or any combination thereof, to name just a few examples. In an embodiment, remote control **110** wirelessly communicates with media device **106**, display device **108** using cellular, Bluetooth, infrared, etc., or any combination thereof.

[0037] Multimedia environment **102** may include a plurality of content servers **120** (also called content providers or sources **120**). Although only one content server **120** is shown in FIG. **1**, in practice the multimedia environment **102** may include any number of content servers **120**. Each content server **120** may be configured to communicate with network **118**.

[0038] Each content server **120** may store content **122** and metadata **124**. Content **122** may include any combination of music, videos, movies, TV programs, multimedia, images, still pictures, text, graphics, gaming applications, advertisements, programming content, public service content, government content, local community content, software, and/or any other content or data objects in electronic form.

[0039] In some embodiments, metadata **124** comprises data about content **122**. For example, metadata **124** may include associated or ancillary information indicating or related to writer, director, producer, composer, artist, actor, summary, chapters, production, history, year, trailers,

alternate versions, related content, applications, and/or any other information pertaining or relating to content **122**. Metadata **124** may also or alternatively include links to any such information pertaining or relating to content **122**. Metadata **124** may also or alternatively include one or more indexes of content **122**, such as but not limited to a trick mode index.

[0040] Multimedia environment **102** may include one or more system servers **126**. System servers **126** may operate to support media device **106** from the cloud. It is noted that the structural and functional aspects of system servers **126** may wholly or partially exist in the same or different ones of system servers **126**.

[0041] Media devices **106** may exist in thousands or millions of media systems **104**.

[0042] Accordingly, media devices **106** may lend themselves to crowdsourcing embodiments and, thus, system servers **126** may include one or more crowdsource servers **128**.

[0043] For example, using information received from media devices **106** in the thousands and millions of media systems **104**, crowdsource server(s) **128** may identify similarities and overlaps between closed captioning requests issued by different users **132** watching a particular movie. Based on such information, crowdsource server(s) **128** may determine that turning closed captioning on may enhance users' viewing experience at particular portions of the movie (for example, when the soundtrack of the movie is difficult to hear), and turning closed captioning off may enhance users' viewing experience at other portions of the movie (for example, when displaying closed captioning obstructs critical visual aspects of the movie). Accordingly, crowdsource server(s) **128** may operate to cause closed captioning to be automatically turned on and/or off during future streaming of the movie.

[0044] System servers **126** may also include an audio command processing module **130**. As noted above, remote control **110** may include a microphone **112**. Microphone **112** may receive audio data from user **132** (as well as other sources, such as display device **108**). In some embodiments, media device **106** may be audio responsive, and the audio data may represent verbal commands from user **132** to control media device **106** as well as other components in media system **104**, such as display device **108**.

[0045] In some embodiments, the audio data received by microphone **112** in remote control **110** is transferred to media device **106**, which is then forwarded to audio command processing module **130** in system servers **126**. Audio command processing module **130** may operate to process and analyze the received audio data to recognize a verbal command from user **132**. Audio command processing module **130** may then forward the verbal command back to media device **106** for processing.

[0046] In some embodiments, the audio data may be alternatively or additionally processed and analyzed by an audio command processing module **216** in media device **106** (see FIG. 2). Media device **106** and system servers **126** may then cooperate to pick one of the verbal commands to process (either the verbal command recognized by audio command processing module **130** in system servers **126**, or the verbal command recognized by audio command processing module **216** in media device **106**).

[0047] FIG. 2 illustrates a block diagram of an example media device **106**, according to some embodiments. Media device **106** may include a streaming module **202**, a processing module **204**, a storage/buffers **208**, and a user interface module **206**. As described above, user interface module **206** may include audio command processing module **216**.

[0048] Media device **106** may also include one or more audio decoders **212** and one or more video decoders **214**.

[0049] Each audio decoder **212** may be configured to decode audio of one or more audio formats, such as but not limited to AAC, HE-AAC, AC3 (Dolby Digital), EAC3 (Dolby Digital Plus), WMA, WAV, PCM, MP3, OGG GSM, FLAC, AU, AIFF, and/or VOX, to name just some examples.

[0050] Similarly, each video decoder **214** may be configured to decode video of one or more video

formats, such as but not limited to MP4 (mp4, m4a, m4v, f4a, m4b, m4r, f4b, mov), 3GP (3gp, 3gp2, 3g2, 3gpp, 3gpp2), OGG (ogg, oga, ogv, ogx), WMV (wmv, wma, asf), WEBM, FLV, AVI, QuickTime, HDV, MXF (OP1a, OP-Atom), MPEG-TS, MPEG-2 PS, MPEG-2 TS, WAV, Broadcast WAV, LXF, GXF, and/or VOB, to name just some examples. Each video decoder **214** may include one or more video codecs, such as but not limited to H.263, H.264, HEV, MPEG1, MPEG2, MPEG-TS, MPEG-4, Theora, 3GP, DV, DVCPRO, DVCProHD, IMX, XDCAM HD, XDCAM HD422, and/or XDCAM EX, to name just some examples.

[0051] Now referring to both FIGS. **1** and **2**, in some embodiments, user **132** may interact with media device **106** via, for example, remote control **110**. For example, user **132** may use remote control **110** to interact with user interface module **206** of media device **106** to select content, such as a movie, TV show, music, book, application, game, etc. Streaming module **202** of media device **106** may request the selected content from content server(s) **120** over network **118**. Content server(s) **120** may transmit the requested content to streaming module **202**. Media device **106** may transmit the received content to display device **108** for playback to user **132**.

[0052] In streaming embodiments, streaming module **202** may transmit the content to display device **108** in real time or near real time as it receives such content from content server(s) **120**. In non-streaming embodiments, media device **106** may store the content received from content server(s) **120** in storage/buffers **208** for later playback on display device **108**.

Clouding Computing System to Perform Computations for a User Device.

[0053] FIG. **3** illustrate an example clouding computing environment **300** to perform computations for a user device, according to some embodiments. Clouding computing environment **300** may include a user device **301**, and a cloud computing system **310** communicatively coupled by a network **317**. Clouding computing environment **300** may be an example of a portion of multimedia environment **102** shown in FIG. **1**.

[0054] In some embodiments, user device **301** may be any device used by a user to perform various computing or communication tasks. For example, user device **301** may be a mobile phone (e.g., a smart phone, a radiotelephone, etc.), a laptop computer, a tablet computer, a handheld computer, a gaming device, a wearable communication device (e.g., a smart wristwatch, a pair of smart eyeglasses, etc.), a desktop computer, a server, an embedded device, or a similar type of device.

[0055] In some embodiments, one or more portions of network **317** may be an ad hoc network, an intranet, an extranet, a virtual private network (VPN), a local area network (LAN), a wireless LAN (WLAN), a wide area network (WAN), a wireless wide area network (WWAN), a metropolitan area network (MAN), a portion of the Internet, a portion of the Public Switched Telephone Network (PSTN), a cellular telephone network, a wireless network, a WiFi network, a WiMax network, any other type of network, or a combination of two or more such networks.

[0056] In some embodiments, cloud computing system **310** includes an environment that delivers computing as a service, whereby shared resources, services, etc. Cloud computing system **310** may provide computation, software, data access, storage, and/or other services that do not require end-user knowledge of a physical location and configuration of a system and/or a device that delivers the services. Cloud computing system **310** may include computer resource **311** and computing resource **321**, in addition to other clouding computing components.

[0057] Each computing resource, e.g., computer resource **311**, computing resource **321**, can include one or more personal computers, workstations, computers, server devices, or other types of computation and/or communication devices, and computing instances executing in computing resources. A computing resources, e.g., computer resource **311**, computing resource **321**, may communicate with other cloud computing resources via wired connections, wireless connections, or a combination of wired or wireless connections.

[0058] In some examples, computer resource **311** may include a group of cloud resources, such as one or more applications (“APPs”) **312**, one or more virtual machines (“VMs”) **313**, virtualized storage (“VS”) **314**, and one or more hypervisors (“HYPs”) **315**. APP **312** may include one or more

software applications that may be provided to or accessed by other components or computing devices, e.g., user device **301**.

[0059] Virtual machine **313** may include a software implementation of a machine (e.g., a computer) that executes programs like a physical machine. Virtual machine **313** may be either a system virtual machine or a process virtual machine, depending upon the use and degree of correspondence to any real machine by virtual machine **313**. A system virtual machine may provide a complete system platform that supports execution of a complete operating system (OS). A process virtual machine may execute a single program and may support a single process. Virtual machine **313** may execute on behalf of a user (e.g., user device **301**), and may manage infrastructure of the cloud computing system **310**, such as data management, synchronization, or long duration data transfers.

[0060] Virtualized storage **314** may include one or more storage systems and/or one or more devices that use virtualization techniques within the storage systems or devices of computing resource **311**. With respect to a storage system, types of virtualizations may include block virtualization and file virtualization. Block virtualization may refer to abstraction (or separation) of logical storage from physical storage so that the storage system may be accessed without regard to physical storage or heterogeneous structure. The separation may permit administrators of the storage system flexibility in how administrators manage storage for end users. File virtualization may eliminate dependencies between data accessed at a file level and location where files are physically store. This may enable optimization of storage use, server consolidation, and/or performance of non-disruptive file migrations.

[0061] Hypervisor **315** may provide hardware virtualization techniques that allow multiple operations systems (e.g., “guest operating systems”) to execute concurrently on a host computer, such as computing resource **311**. Hypervisor **315** may present a virtual operating platform to the guest operating systems, and may manage the execution of the guest operating systems multiple instances of a variety of operating systems and may share virtualized hardware resource.

[0062] Computing resource **321** can be an example of computing resource **311**, and can include various machines, e.g., machine **322**, which can be a computing device or a virtual machine, e.g., VM **113**, and can include a processor **323** and a memory **324**. Computing resource **321** can include storage **328**, and other components, not shown.

[0063] User device **301** may include a display **303**, a storage **302**, a processor **304**, an operating system (OS) **306**, and an input device **308**. A web browser **305** may operate on user device **301**. A user rendered context **319**, which is a part of a user application **320**, can be stored in storage **302**. User rendered context **319** can provide input/output (I/O) functions for user application **320**. An application user interface (UI) **309** for user rendered context **319** can be presented on display **303** within a user renderer frame **307**. In some embodiments, when user device **301** does not have web browser **305** installed, user renderer frame **307** can be the top level window of the graphical user interface (GUI) for user application **320**. In some other embodiments, when user device **301** has web browser **305** installed, user renderer frame **307** can be contained within the root window of the GUI for web browser **305**.

[0064] User application **320** can include user rendered context **319** and user programmer context **331** that operates within cloud computing system **310** supported by computing resource **321**. Computing resource **321** can support a container **332** that contains multiple user programmer contexts, such as user programmer context **331**, user programmer context **333**, user programmer context **335**, and more. In some examples, there can be thousands or more different user programmer contexts included in container **332**. Each user programmer context can be for a different application with a user rendered context operating on a different user device.

[0065] User application **320** includes user rendered context **319** operating on user device **301** and user programmer context **331** operating in cloud computing system **310**. However, user application **320** can be different from a web application where part of the code of a web application can run in a browser on a user device, with some backend code to perform some operations at a web server. A

web application normally is a collection of different parts. The part operating at the web browser on a user device can be written in different languages such as HyperText Markup Language (HTML), Cascading Style Sheets (CSS), or Javascript. The part of a web application written by HTML can function without other parts. In comparison, user application **320** can be written in one language for both user rendered context **319** and user programmer context **331**. In addition, as shown in FIG. 7, user rendered context **319** cannot start operation until user programmer context **331** is initialized. Therefore, user rendered context **319** and user programmer context **331** are integrated together, one cannot function without another, instead of a collection of different functional parts as in a web application. Furthermore, when a web application that undergoes page transitions, the web server sends down HTML to update or replace a current page or part thereof. In other words, the web server computes the results of what the updated web page should be, and the user device merely layout the received updated web page or a part of it from the web server. On the other hand, in some embodiments, the user programmer context **331** in the cloud computing system computes a difference tree to represent the differences between the updated tree and the current tree, while user rendered context **319** operating on user device **301** computes the final tree before performing the layout of the updated tree. Moreover, the difference tree sent from the cloud computing system to user device **301** can be represented by JSON instead of HTML.

[0066] FIG. 4 illustrates an example software process **400** for developing an application operating on a user device and a cloud computing system, e.g., user application **320**, according to some embodiments. In some examples, an application goes through multiple stages from the time a developer team starts to program user application **320** to the time a user can use user application **320**.

[0067] At code stage **401**, user application **320** is programmed. The code for user application **320** can also include code from existing libraries from the platform or from other third parties. The code for user application **320** and the library code can be bundled and compiled at build stage **403**. After being built, at deployment **405**, user application **320** is ready for deployment to a server in a cloud computing system and a client, which can be user device **301**. For example, user application **320** can be deployed to a SaaS server. At runtime stage **407**, user application **320** can be operated on user device **301** to load user rendered context **319** onto user device **301**, and load user programmer context **331** on a server of cloud computing system **310**.

[0068] In some embodiments, some system-supplied code exists in the form of Javascript (JS) libraries that are packaged and distributed via node package manager (npm). This system-supplied code can provide a high-level interface and abstraction to the underlying host architecture which includes Roku OS, or other browsers and possibly other TV OS at user device **301**. The libraries also include a React-like library for writing components, business logic and control flow, and a client-side library for writing GUI components like lists, grids and dialogs. There is also a complete component library containing standard GUI components like sliding panels, row list, or other GUI components known to one having ordinary skills in the art.

[0069] During development at code stage **401**, the system-supplied code and user developed code can be bundled together and can be debugged in a browser. Furthermore, the developer is free to include code from other sources (other libraries, other components etc), eschew using the system libraries or parts thereof and even modify (fork) system-supplied code to customize it. In many respects, the user code may bring its own libraries and components, which may exceed the normal capability provided in a web browser development environment.

[0070] During build time at build stage **403**, the various packages of code can be bundled and uploaded to a server in cloud computing system **310** where they are available for further testing on device, and ultimately used in production.

[0071] There is also a mechanism by which a system provider can deploy trusted code to implement important Platform APIs like RokuPay. Here, the code cannot be seen or inspected by the developer, and the developer has to trust that it has not been modified at build time or altered at

runtime.

[0072] FIGS. 5-6 illustrate example architectural overviews of applications operating on a user device and a cloud computing system, according to some embodiments.

[0073] FIG. 5 shows an abstract view of system architecture **500** that includes a user application **520**, which includes user rendered context **519** and user programmer context **531** operating on a user device and a cloud computing system. User programmer context **531** can be operated on a VM **504** within a cloud computing system. User rendered context **519** can be operated on a user device including hardware platform **501**, an OS **503**, a software platform **505** that can include an engine **507** and various other components **509**.

[0074] FIG. 6 illustrates more details of a system architecture **600**. System architecture **600** can be an example of system architecture **500**, with more details on some of the components. In some examples, system architecture **600** can be referred to as Opus. Advantages of Opus may include: (1) Roku Apps can run in the cloud computing system **310**, and UI is rendered on user device **301**; (2) Opus system architecture can support much faster development cycles by allowing use of modern cloud technologies (e.g. node.js, javascript, etc.) to write Apps; (3) Opus system architecture can use standard programming languages, tools and libraries, such as Javascript or WebAssembly (WASM); (4) User applications can be developed and debugged in a browser; (5) User apps written for Opus can be deployed on non-Roku OS targets; (6) User apps can provide Channel Store support; (7) Opus system architecture can support all relevant Roku APIs such as Roku Pay, Roku Advertising Framework (RAF), authentication, or APIs from any other third parties, etc.

[0075] System architecture **600** includes user rendered context **319** operating on user device **301** and user programmer context **331** operating in cloud computing system **310**. In addition, system architecture **600** can include other private environments, such as private program of service, private context, master context, and more.

[0076] User programmer context **331** is the part of application **320** that can be executed in cloud computing system **310** or directly on hardware if it is powerful enough. It is responsible for the business logic, application flow, data model and responses to user navigation.

[0077] User renderer context **319** executes on user device **301** and is responsible for drawing to the screen and receiving and reacting to user input such as keystrokes. In system architecture **600**, application developers can write their own components, also in Javascript, and have them perform highly stylized animations or effects. A pre-built set of components, libraries and patterns can also be supplied for developers to use or extend.

[0078] System architecture **600** can also include one more private Contexts. There are several private contexts that also execute JS or WASM code. These are discussed in a section below.

[0079] User renderer context **319** can include many features. User renderer context **319** maintains a traditional render tree composed of a tree of nodes. In some embodiments, there are no complex class hierarchies among the nodes of the trees. Instead, each node is a pure JS object, the simplest of which is just { }. Each node may have a collection of properties, stored in property attributes (“props”). Each node may have a collection of child nodes stored as an array on the node, stored in children. Each node may have a tag stored in the property element. Text nodes are represented as a raw string inside a child array.

[0080] User renderer context **319** can generate drawing instructions to an abstracted 2D drawing surface using an API that adheres to a subset of the HTML 2D Canvas specification. This is the primary way in which user renderer context **319** works in a web browser and on Roku OS hardware. However, user renderer context **319** does not need to run HTML in a browser. Instead, user renderer context **319** works the same when user device **301** supports only Javascript and a suitable 2D drawing interface (plus a few other base requirements). User renderer context **319** also manages the keyboard focus, dispatches key events and manages update operations on the node tree. User renderer context **319** does not perform the computation to generate the render tree. The

render tree is generated by user programmer context **331** instead. User programmer context **331** generates the render tree, or an update to the render tree, sends the update of the render tree to user rendered context **319**, and user rendered context **319** performs the update to the render tree.

[0081] User renderer context **319** includes small components. A Small Component is responsible for low-level behavior such as key responses, animations and graphical appearance. A Small Component is to distinguish from high-order components that run in user programmer context **331**. If a node of the render tree has an element property, this identifies a collection of JS functions that will run in the renderer context at various phases of the drawing, mounting or update algorithms. For example, a tag called “rectangle” can be defined for the render tree, this will reference a dictionary of JS functions known to the renderer, as shown below:

```
TABLE-US-00001    const rectangle = {    draw: (...) => {...},    handleKeyEvent: (...) => {...},    onComponentDidMount: (...) => {...} };
```

[0082] The base renderer does not contain any of its own small components, all of these may be provided by the application programmer although there will be one or more component libraries that define and implement a particular widget style and are available for use out-of-the-box.

[0083] Once a node has been inserted into the render tree (known as mounting) by user rendered context **319**, the render tree may keep and reference internal state variables. These variables are used to track things such as currently selected item or animation transition effects. Internal state is local to user rendered context **319** only, and may not be transmitted to user programmer context **331**. For example, consider a Modal Dialog component. This component may have a question, along with two possible actions from the user, OK or Cancel. While the model is displayed, internal state is used to track which response is currently selected while the Left and Right keys can be used to alternate between those responses. Only when the user selects an action, is the result transmitted user programmer context **331**. Small Components can attach a state object to the node to keep track of these values.

[0084] The mounting pass—at the beginning of a session, user rendered context **319** is initialized with a render tree that has been provided from user programmer context **331**. User rendered context **319** can visit each node in the render tree and call the lifecycle function `onComponentDidMount` for each component that has one. In some examples, `onComponentDidMount` is called when a node is first added to the tree. It is up to the small component for that node to do anything (it might do nothing). Typically, the component would establish default state variables. During this pass, the components have chance to initialize internal state. For example, if a component is managing a list of items that can be selected, it may initialize a state variable to indicate which item is currently selected. This state variable can be adjusted later by an event such as a keypress. Components that have animations would typically create state variables to manage the current state of animation. Components can also make pre-calculations at this point as well as examine the node's properties and any children it has.

The Drawing Pass

[0085] The positioning of each node of the render tree is relative to the parent, and the width and height are pre-calculated by the parent as well. To help with this, the base renderer drawing algorithm can adjust the canvas origin to (0, 0) for each child node. It also has built-in interpretations for the properties:—left, right, top, bottom, width, height and bounds. For example, if a node has property `left={30}`, the drawing algorithm adjusts the origin such that drawing at (0,0) will draw 30 pixels to the right of the parent's origin. The width of this child control would also be adjusted to be 30 px less than the parent.

[0086] In some embodiments, a node's placement rectangle can be contained within its parent. However, the child nodes may use negative numbers to adjust its rectangle to be expanded beyond that of its parent. Provided the parent has not clipped its children, that would allow children to draw outside of their parents' rectangle.

[0087] User rendered context **319** can have some simple drawing algorithm. Starting from the root

of the tree, user rendered context **319** performs a breadth-first iteration of the tree, giving each node chance to issue draw instructions. By default, the algorithm will visit the entire tree, but this can be trimmed by components if they know that their children are off screen.

[0088] When visiting a node, the drawing algorithm adjusts the origin, calculates the width and height of the node, and then delegates drawing responsibility to the component's draw function (if it has one). In a drawing function, a component can perform the following operations: (1) issue drawing commands to the drawing context; (2) push and pop state to the drawing context using `save ( )` `restore ( )` and other mutating methods such as `fillStyle`; (3) clip the drawing rectangle to contain this node (and its children) within a specified rectangle. In some examples, `fillStyle` is a property of the drawing service that is used in subsequent calls. For example, if `fillStyle` is “solid” then `fillRectangle` will draw a solid rectangle. The component can further manage drawing of the children by adjusting the origin before drawing each child, selectively drawing children that are visible and skipping those that are not, and delegating drawing commands to child nodes or drawing them directly. In addition, the component can further perform timing calculations related to the current frame time value (required to handle animation effects, and determine when they end).

[0089] User rendered context **319** can also keep track of a “next frame requested” time that can be set by any component. When a component is animating, it will request an immediate frame callback to make sure it can be animated smoothly. When no components request a callback, the render loop can idle until an external event starts the draw loop again. The entire tree is rendered on each frame-user rendered context **319** does not make an attempt to optimize this by only redrawing dirty areas of the screen.

[0090] Drawing command list—The drawing undertaken by user renderer context or private context can actually be made to a proxy Canvas object implementing the HTML 5 Canvas API. Underneath, that proxy writes the commands to a list. Once done, the list is sent to the Master Context where the actual drawing takes place. This is done to isolate drawing in one context from another. It also allows better control flow and timing measurements since the CPU required to draw the frame will not be interleaved with the CPU the renderer is using in its drawing logic (it gets queued and done at the end). Another benefit is to take advantage the extra processor cores available on some devices.

[0091] Updates—an update is a transaction that arrives from user programmer context **331** in response to an event, effect or timeout. The update record will contain a tree diff that contains instructions for amending the currently active render tree. The tree diff may be referred to as a difference tree. Updates can result in a few different situations, such as: (1) an existing node or subtree being removed from the graph (unmounted); (2) a new node or subtree being added as a child to an existing component (mounted), (3) a child taking a new position in parent's children list (moved), or (4) a node receiving updated props, either changes, additions, or props being deleted (updated)

[0092] An update operation can be sweeping and involve many nodes, or it could consist of just one node being updated. User programmer context **331** can produce a diff without the need for user rendered context **319** to send its existing tree, because user programmer context **331** can keep a copy of the existing tree on user rendered context **319**. The diff is minimally represented on the wire, and can be safely applied as a transaction to the current tree. A diff represents changes that need to be made to the current existing tree of user rendered context **319**.

[0093] The above described update operations are different from the traditional Document Object Model (DOM) architectures used in HTML and SceneGraph. In a traditional DOM architecture, user programmer context **331** can perform direct operations on the render tree, make changes to nodes as well as respond to events by further modifying the render tree. This can happen at any time and can introduce cryptic edge cases or make it difficult to achieve consistency. In embodiments, user programmer context **331** cannot make changes directly on the render tree

managed by user rendered context **319**. Instead, user programmer context **331** can only send any update operation in a transaction to user rendered context **319** so that user rendered context **319** can update the render tree based on the information from user programmer context **331**. In Opus, the update operation can separate user rendered context **319** from user programmer context **331** in its entirety (there is no backdoor way to read from the renderer). In addition, user programmer context **331** is separated by a transactional request/response transport so that it can be run in the cloud.

[0094] In embodiments, user rendered context **319** is single threaded and each of its tree traversing algorithms is sequential, an update operation cannot be interrupted by an event or interrupt the drawing algorithm. It means that an update operation is guaranteed to be applied safely to the tree.

[0095] Client-side events—client-side events are those generated by the local system at user device **301**, such as key presses. The client-side is the user device side, with respect to the server side which is the cloud computing system. These are sent into user rendered context **319** and may be routed into handlers within the node tree. For example, a key event may be routed to a component via its `handleKeyEvent` entry point. Each client-side event handler may only do the following things: (1) change internal state variables, (2) delegate the event to one or more children (provided they follow these same rules), (3) bubble an event to its parent, or (4) fire an external event.

[0096] Notably, event handlers should not change props, children or element properties of any node in the tree (which would change its structure and break everything).

[0097] External events—an external event is one that is generated by a component directed at user programmer context **331**. For example, using the Model Dialog example, once the user has selected a response, the component may send an `onChoiceSelected` event. In some examples, `onChoiceSelected` is an example name for an external event. In this example, the event might cause the user program to dismiss the modal dialog with an affirmative or negative response from the user. Normally, user programmer context **331** would respond with an update transaction. To identify the component, there is a special property on each relevant node called `id`. The semantics, name and parameters of external events are up to the component designer and form part of its contract.

[0098] Effects and timeouts—there are two special cases that user rendered context **319** can callback to user programmer context **331**. A special effect event is fired only after an update transaction has been applied to the tree. This guarantees that all nodes have been mounted and allows user programmer context **331** to run any side-effects associated with any of those nodes. Embodiments herein can achieve things like Ajax calls or any other asynchronous operations. An update transaction may include a single timeout value which is a timestamp that will occur in the future. Assuming that no other client-side events are fired, user rendered context **319** will fire an empty event when the timeout has expired. The timeout will normally execute some sort of update such as changing the currently selected item in a carousel.

User Programmer Context

[0099] One of the main goals of Opus is to execute user programmer context **331** in the cloud computing system **310** in a way that does not require a permanent instance of the program to be running on a server. This can be achieved by using functional programming techniques.

Components and state can be expressed using pure functions and so forms the basis for a session-less solution.

[0100] Generating trees using React and JSX—in Opus, the user programmer context **331** can generate and update the render tree. It may generate an initial tree; and (2) process an event and generate an updated tree.

[0101] If user programmer context **331** is deterministic (always generates the same tree with the same inputs and same event sequence), user programmer context **331** can create an efficient diffing algorithm. The diffing algorithm can be run against the original tree and an updated tree to generate a diff which can then be sent to user rendered context **319** in an update operation. This means the programmer does not need to write any mutating logic at all for user rendered context **319**,

transactions are created by user programmer context **331** and transferred over the wire or connection to user rendered context **319**.

[0102] In addition, in some embodiments, since generating trees can be repeated by executing the program with the same parameters, user programmer context **331** may not save the trees. Instead, user programmer context **331** can re-generate a tree when needed by supplying the same parameters (state).

[0103] The following is a simple program: [0104] `const tree=render (<div>Hello World</div>);`

[0105] (JSX is pure JS syntax, with an extension that understands HTML-like tags which are turned into createElement function calls.

[0106] The generated tree will be in the form of JSON that can be transmitted to the renderer. In this example, that tree would be:

TABLE-US-00002 { children: [“Hello World”], element: “div”);

[0107] User programmer context **331** can include Big Components. A component can be similar to the component concept defined in React Javascript library (“React”). The following is such an example of a component.

TABLE-US-00003 `const Greeting = ({name}) => <div>Hello {name}</div>; const tree = render(<Greeting name=“Janet”/>); //produces {    children: [“Hello Janet”],    element: “div” }`

[0108] The component Greeting shown above is called a Big Component because it may start with a capital letter. Notice that tag names that start with small letters become part of the final render tree and are not given any special meaning by user programmer context **331**. Such final render tree can be referred to as a Small Component which are ones provided for user rendered context **319**.

For example:

TABLE-US-00004 `const Greeting = ({name}) => <div color=“blue”><greeting name={name}/></div>; const tree = render(<Greeting name=“Janet”/>); //produces { props: { color: “blue” }, children: [{ props: { name: “Janet” }, element: “greeting” }], element: “div” }`

[0109] Notice that a Big Component definition is actually a function. With respect to React, the rendering algorithm traverses the element tree executing (or reconciling) the big components, and results in a tree that only references small components.

Functional State Management

[0110] Traditional applications collect state as the program runs and that state cannot be detached from the program. In embodiments, a state hook is used which allows state to be written functionally to achieve a session-less server architecture using the principle that the same function executed with the same input produces the same output. The state hook looks like this:

TABLE-US-00005 `const React, { useState } from “<path to react library>”; const Counter = ( ) => {    const [count, setCount] = useState(0);    const increaseCount = ( ) => setCount(count + 1); return <div>    <div>Current count: {count}</div>    <button onSelect={increaseCount}>Increase</button> </div>;`

[0111] The state hook requires user programmer context **331** to explicitly declare state variables using useState () function, and the only way to ‘modify values is through the setCount function returned by the hook.

[0112] FIGS. 7-8 illustrate example sequence diagram of operations performed by an application operating on a user device, according to some embodiments. The following descriptions explain operations in FIGS. 7-8.

[0113] Events—the above example also shows how an event can be associated with a small component. Here, onSelect is an external event supported by a button small component. When the user selects the button, user rendered context **319** generates an external event which in turns makes a request to user programmer context **331**. The framework can make the event be routed to the event handler. Once the event is executed, the reconciler of the framework can render the entire program again using the new state values. Finally, the reconciler can diff the original tree (before

the event was executed) with the new one and generate an update record to send to the renderer. In some examples, the steps can include: (i) framework creates a copy of the render tree using the current state (this results in a copy of the tree identical to the one on the client); (ii) the framework executes the event handler which changes the state; (iii) the framework generates another copy of the render tree using the new state; and (iv) frameworks determines the difference between those two trees and sends back the difference to the user renderer context. Embodiments herein can be different from a normal React implementation. For example, in a normal React implementations, step (i) may not be needed because the program context is always kept in memory and has an affinity with the user renderer context. In some embodiments herein, the user program context can recreate the tree from the state when needed instead of saving the complete tree all the time.

[0114] The renderer and program need to know what the state values are at all times. To facilitate this, the Program may attach a state reference to the update transaction. This is similar to a cookie or a session identifier, except it may represent the immutable serialized state of the Program. When the renderer applies the transaction, it stores the state reference. Future events supply the state reference as a parameter in the event payload.

[0115] Asynchronous Effects-asynchronous effects can be demonstrated in FIG. 8.

[0116] In some embodiments, the event handlers can be synchronous once the fireEvent call reaches the server. This means that the event handlers cannot be asynchronous functions or generate side effects that call the setState modification functions in a callback. If this were the case, the fireEvent call could not be completed until all the asynchronous calls are resolved and this would leave us unable to respond to events in a timely manner.

[0117] React achieves asynchronous behavior through the useEffect hook. For example, the following code segment can demonstrate the asynchronous behavior. The code works in standard React because user programmer context 331 stays running and connected to the state setting function (i.e. setCards () returned by the useState hook. React can then resolve the call and trigger an update transaction on the actual tree. In some examples, user programmer context 331 may not stay running and cannot receive results from service calls or timing events.

[0118] In some embodiments, the service here as shown in FIG. 8 may just be an example of an external service call. In Javascript, external service call can be done asynchronously—the thread cannot block waiting for the result of the external service call. The service in this example is an arbitrary web service external to the system.

```
TABLE-US-00006 import <path to react library>, { useEffect, useState } from “<path to react library>”; const fetchJson = async url => { const response = await fetch(url); return response.json( ); }; export const Ajax = ( ) => { const [cards, setCards] = useState([ ]); useEffect(( ) => { (async ( ) => { const { deck_id: deckId } = await fetchJson(“https://<my-cards-api>/api/deck/new /shuffle/?deck_count=1”); const { cards } = await fetchJson(‘https://<my-cards-api>/api/deck/${deckId}/draw/? count=2’); setCards(cards); } ) ( ); //can return a ‘remove sideeffect’ function, which in this case would cancel the async operation if it is still pending }, [ ]); const [card, card2] = cards; return ( <panel key=“ajax”> {!card && “Dealing two cards...”} {card && “You got...”} {card && <poster top={40} left={90} uri={card.image} />} {card2 && <poster top={70} left={120} uri={card2.image} />} </panel> );
```

[0119] However, in Opus, the state is disconnected from user programmer context 331 and so the setCards () call cannot be valid in this context. To remedy this, embodiments herein introduce a hook with similar semantics to useEffect, useAsyncEffect () which declares an asynchronous function to be executed once the component has been mounted by user rendered context 319. The following usage would replace useEffect in the above example:

```
TABLE-US-00007 useAsyncEffect(async ( ) => { const { deck_id: deckId } = await fetchJson(“https://<my-cards-api>/api/deck/new/shuffle/? deck_count=1”); const { cards } = await fetchJson(‘https://<my-cards-api>/api/deck/${deckId}/draw/?count=2’); setCards(cards);
```

//NOTE: cannot support remove side effect semantics }, []);

[0120] To implement this, the response payload contains a flag `hasEffects` which is set to true if there is at least one unresolved use `AsynEffect` pending in the current response. On the renderer side, if any response has `hasEffects` set to true, the renderer will immediately issue a `fireEffects` call to the server (this is similar to sending an empty fire event call). The `fireEffects` call can be long running and the server is free to execute all asynchronous callbacks due and only return the result to the renderer when they have all been resolved.

[0121] One caveat is that if the renderer generates a new external event while `fireEffects` is pending, the `fireEffects` call may be abandoned to let the `fireEvent` call go through, and then if it is still relevant, it would be tried again after the event was resolved.

[0122] Private Contexts—to facilitate Platform APIs such as Roku Pay and RAF, Opus includes the concept of Private Contexts. This is primarily a client side JS isolate, or sandbox, into which Roku can load libraries that need to be kept private so they can be trusted. The initialization of these libraries is a platform feature in itself. At the low level, it includes a way to insert a private component into a render tree (if the library supports a GUI like the Roku Pay purchasing flow) or a way to call a private function directly if it does not require a GUI.

[0123] In some embodiments, a Private Platform API can be shipped with a public library that defines its interface. By way of example, this is how a Roku Pay order dialog might be launched by a user application:

```
TABLE-US-00008 import React, { useState } from "<path to react library>"; import {
ChannelStore, Order } from "@roku/roku-pay"; import { Modal } from "@roku/components";
const BuyProduct = ({ productCode, onSuccess, onFailure }) => {  const [state, setState] =
useState("ordering");  const handleOrderStatus = result => { if(result.success) {
setState("ok"); } else { setState("canceled"); } } if(state === "ordering") { return <ChannelStore
command="doOrder" onOrderStatus={handleOrderStatus}> <Order code={productCode} qty=
{1}> Fluffy Slippers </Order> </ChannelStore> } if(state === "ok") { return <Modal onClose=
{onSuccess}> Thank you. Enjoy your slippers... </Modal> } if(state === "canceled") { return
<Modal onClose={onFailure}> Got cold feet? ... </Modal> }
```

[0124] The implementation of `ChannelStore` might look like this:

```
TABLE-US-00009 export const ChannelStore = ({ children, ...props }) => <isolate
guid="<CHANNEL-STORE- GUID>" ...props>{children}</isolate>; export const Order = ({
children, code, qty }) => <order code={code} qty={qty}>{children}</order>;
```

`isolate` is a special small component that the user renderer interprets as a placeholder for a private component. When asked to mount such a component, the user context passes the parameters up to the Master Context which is able to resolve the Globally Unique Identifier GUID (or URI) to identify the `ChannelStore` library.

[0125] The Master Context then loads `ChannelStore` into a private isolate and initializes it with the render sub-tree. From that point on, the Master Context will marshal events between the two contexts and be responsible for directing the key events to the active context, as well as merging the drawing instructions from both contexts to produce one frame.

Techniques for Reducing Time for Drawing and Rendering User Interfaces

[0126] In some embodiments, GUI elements include a plurality of sub-elements. For instance, a GUI element (e.g., a tile) may include a background, text, an image, a progress bar, one or more informational icons, etc. One or more of such sub-elements may be layered on top of the other, and each of these sub-elements may be rendered based on a respective drawing command or instruction specific to the sub-element. The layering of such sub-elements can reduce the frame rate, as each frame of the GUI takes longer to render depending on the complexity (e.g., based on the number of sub-elements and corresponding drawings instructions) of a GUI element to be rendered. This can lead to choppy animation, which can make such devices less appealing to use.

[0127] The techniques described herein mitigate such issues. For instance, if certain UI elements

comprise many sub-elements, a flattened representation of such UI elements are generated and cached. This way, the next time such UI elements are to be rendered and displayed in a GUI, the cached representations are retrieved and displayed instead of executing the drawing instructions for such UI elements. By doing so, the functioning of a computing device on which such a GUI executes is improved, as the number of drawing instructions to be executed for the GUI is reduced. This advantageously reduces the number of compute resources (e.g., processing cycles, I/O, memory, power, etc.) that are utilized to display the GUI. Moreover, the user experience is improved, as the frame rate of the GUI is not reduced.

[0128] For example, FIG. 9 is a block diagram of a media device **900** configured to optimize the rendering of a GUI, according to some embodiments. As shown in FIG. 9, media device **900** includes a user rendered context **919**, a user programmer context **931**, and a memory **916**. User rendered context **919** is an example of user rendered context **319** and **519**, and user programmer context **931** is an example of user programmer context **331** and **531**, as respectively described above with reference to FIGS. 3 and 5-8. In the embodiment shown in FIG. 9, both user rendered context **919** and user programmer context **931** execute on media device **900**. However, the embodiments described herein are not so limited. For example, user programmer context **931** may be configured to execute on a cloud computing system, as described above with reference to FIGS. 3 and 5-8. Media device **900** is an example of media device **106** and user device **301**, as described above with reference to FIGS. 1, 2, 3, and 6-9. Examples of memory **916** include, but are not limited to, dynamic random access memory (DRAM), synchronous DRAM (SDRAM), double data rate (DDR, DDR2, DDR3, etc.), RAMBUS DRAM (RDRAM), static RAM (SRAM) or a combination thereof.

[0129] As further shown in FIG. 9, user renderer context **919** (also referred to as a “frontside” component) comprises a user input determiner **902** and a GUI renderer **904**. GUI renderer **904** may comprise a complexity determiner **906** and a hash engine **910**. User programmer context **931** (also referred to as a “backside” component) comprises a data retriever **914**. GUI renderer **902** may be configured to render UI elements of a GUI. For example, GUI renderer **902** may execute drawing instructions for such UI elements. Execution of such drawing instructions may cause the instructions be converted to low-level graphics calls. The low-level graphics calls may be direct commands to a processor (e.g., a central processing unit (CPU), a graphics processing unit (GPU), etc.) to display images and graphics.

[0130] The UI elements that are rendered for the GUI may depend on user input (e.g., keystrokes, interactions with a particular button of a remote control of user device **900** and configured to control the GUI, etc.). For instance, as the user navigates a GUI (e.g., scrolls up/down, interacts with certain UI elements, navigates to different screens, etc.), different UI elements may be presented to the user. User input determiner **902** may be configured to detect user such input and provide an indication of the user input to GUI renderer **904**. Based on the detected user input, GUI renderer **904** may determine whether the render tree is to be updated (as described above with reference to FIGS. 6-8) and/or determine which UI elements (and sub-elements thereof) of the GUI are to be rendered. User programmer context **931** may determine whether such elements and sub-elements are already represented by the current render tree. If such elements and sub-elements are not represented by the current render tree, data retriever **914** of user programmer context **931** may retrieve such items from a web service (e.g., service **807**) or server configured to provide such items. Upon retrieving such items, data retriever **914** may provide an updated render tree and/or the items to user renderer context **919** for rendering thereby. User programmer context **931** may also provide corresponding drawing instructions for such items to user renderer context **919**.

[0131] Complexity determiner **906** may be configured to determine whether drawing instructions for a GUI element are relatively complex. The complexity of a GUI element may be based on the number of drawing instructions required for rendering the GUI element, where greater the number of drawing instructions, the greater the complexity. For instance, suppose a first GUI element

comprises 5 sub-elements, and a second GUI element comprises 2 sub-elements. In such an example, the first GUI element may be considered to be more complex than the second UI element. In some embodiments, complexity determiner **906** compares the number of drawing instructions to a predetermined threshold. If the number of drawing instructions meets and/or exceeds a predetermined threshold, complexity determiner **906** may determine that the corresponding GUI element is relatively complex. Otherwise, complexity determiner **906** may determine that the corresponding GUI element is relatively simple. If complexity determiner **906** determines that a GUI element is relatively simple, complexity determiner **906** may provide the drawing instructions to GUI renderer **904**. GUI renderer **904** may convert the drawing instructions to low-level graphics calls to generate a representation representative of the GUI element (and its sub-elements). GUI renderer **904** may render the representation for the GUI element.

[0132] If complexity determiner **906** determines that a GUI element is relatively complex, GUI renderer **904** may determine whether a representation of the GUI element is already cached in memory **916**. If the representation is already cached, complexity determiner **906** may retrieve the representation, and GUI renderer **904** may render the obtained representation without having to execute any drawing instructions corresponding to the GUI element. Additional details for determining whether a representation of a GUI element is already cached are described below with reference to hash engine **910**. If the representation is not cached, complexity determiner **906** may execute the drawing instructions corresponding to the GUI element, render the representation, and cache the representation in memory **916**.

[0133] In some embodiments, the representation is a bitmap image representative of the GUI element. Memory **916** may be configured to cache representations (shown as representations **918**) for different GUI elements that are generated by GUI renderer **904**.

[0134] GUI renderer **904** may comprise a hash engine **910**. Hash engine **910** may comprise a hash function. Hash engine **910** may receive a set of drawings instructions for rendering a particular UI element as an input. Hash engine **910** may generate a hash value representative of the set of drawing instructions. Hash engine **910** may store the hash value in memory **916**. Memory **916** may be configured to cache hash values (shown as hash values **920**) generated by hash engine **910** for different sets of drawings instructions, each for rendering a particular UI element. In some embodiment, hash engine **910** generates a hash value for sets of drawings instructions that are determined to be relatively complex by complexity determiner **906**.

[0135] If complexity determiner **906** determines that the GUI element is relatively complex, complexity determiner **906** provides an indication to hash engine **910** indicating as such. Hash engine **910** may provide the drawing instructions as an input to the hash function, which generates a hash value representative of the drawing instructions. Hash engine **910** may compare the generated hash value to the hash values stored in memory **916**. If the generated hash value matches one of hash values **920**, then hash engine **910** determines that a representation corresponding to the drawing instructions is already cached in memory **916**. In response to such a determination, hash engine **910** may retrieve the cached representation from memory **916**. GUI renderer **904** may render the obtained representation via the GUI without having to execute the drawing instructions. That is, GUI renderer **904** bypasses execution of the drawing instructions because a representation corresponding to the GUI element is already available for rendering. If the generated hash value does not match any of hash values **920**, then hash engine **920** determines that a representation corresponding to the drawing instructions are not cached in memory **916**. In response, GUI renderer **904** may execute the drawing instructions and render the resulting representation of the GUI element. GUI renderer **902** may also cache the generated representation in memory **916**.

[0136] FIG. **10** is a flowchart for a method **1000** for rendering a cached representation of a UI element for a GUI, according to some embodiments. Method **1000** can be performed by processing logic that can comprise hardware (e.g., circuitry, dedicated logic, programmable logic, microcode, etc.), software (e.g., instructions executing on a processing device), or a combination thereof. It is

to be appreciated that not all steps may be needed to perform the disclosure provided herein. Further, some of the steps may be performed simultaneously, or in a different order than shown in FIG. 10, as will be understood by a person of ordinary skill in the art.

[0137] Method **1000** shall be described with reference to media device **900** of FIG. 9. However, method **1000** is not limited to that example embodiment.

[0138] In **1002**, GUI renderer **904** may obtain a first set of drawing instructions configured to render a first UI element for a GUI. For example, the first set of drawing instructions may be provided by user programmer context **931**.

[0139] In **1004**, GUI renderer **904** may determine that a representation of the first UI element is stored in memory **916**. In some embodiments, the representation of the first UI element comprises a bitmap image representative of the first UI element. Additional details regarding determining that the representation of the first UI element is stored in memory **916** are provided below with reference to FIGS. 11 and 12.

[0140] Operations **1006**, **1008**, and **1010** may be performed in response to determining that the representation of the first UI element is stored in memory **916**. For instance, in **1006**, GUI renderer **904** may bypass execution of the first set of drawing instructions. In **1008**, GUI renderer **904** may obtain the representation of the first UI element from memory **916** based on bypassing execution of the first set of drawing instructions. In **1010**, GUI renderer **904** may render the obtained representation of the first UI element for the GUI.

[0141] FIG. 11 is a flowchart for a method **1100** for determining whether a representation of a first UI element is stored in memory, according to some embodiments. Method **1100** can be performed by processing logic that can comprise hardware (e.g., circuitry, dedicated logic, programmable logic, microcode, etc.), software (e.g., instructions executing on a processing device), or a combination thereof. It is to be appreciated that not all steps may be needed to perform the disclosure provided herein. Further, some of the steps may be performed simultaneously, or in a different order than shown in FIG. 11, as will be understood by a person of ordinary skill in the art.

[0142] Method **1100** shall be described with reference to media device **900** of FIG. 9. However, method **1100** is not limited to that example embodiment.

[0143] In **1102**, hash engine **910** may generate a hash value based on the first set of drawing instructions. For example, the first set of drawing instructions may be provided as an input to a hash function of hash engine **910**, which outputs the hash value based on the first set of drawing instructions.

[0144] In **1104**, hash engine **910** may compare the hash value to a set of hash values **920** stored in memory **916**, wherein each hash value of the set of hash values **920** corresponds to a respective set of drawing instructions for rendering a particular user interface element.

[0145] In **1106**, hash engine **910** may determine that the hash value matches a hash value from the set of hash values **920**.

[0146] In **1108**, hash engine **910** may determine that the representation of the second user interface element is stored in memory **916** based on determining the hash value matches the hash value from the set of hash values **920**.

[0147] FIG. 12 is a flowchart for a method **1200** for determining whether a representation of a first UI element is stored in memory, according to some embodiments. Method **1200** can be performed by processing logic that can comprise hardware (e.g., circuitry, dedicated logic, programmable logic, microcode, etc.), software (e.g., instructions executing on a processing device), or a combination thereof. It is to be appreciated that not all steps may be needed to perform the disclosure provided herein. Further, some of the steps may be performed simultaneously, or in a different order than shown in FIG. 12, as will be understood by a person of ordinary skill in the art.

[0148] Method **1200** shall be described with reference to media device **900** of FIG. 9. However, method **1200** is not limited to that example embodiment.

[0149] In **1202**, complexity determiner **906** may determine that a number of drawing instructions in

the first set of drawing instructions exceeds a predetermined threshold.

[0150] In **1204**, complexity determiner **906** may determine that the representation of the first UI element is stored in memory **916** based on determining that the number of drawing instructions in the first set of drawing instructions exceeds a predetermined threshold. As described above, in some embodiments, representations of UI elements that are determined to be relatively complex are cached in memory **916**, whereas representations of UI elements that are determined to be relatively simple are not cached in memory **916**. Accordingly, complexity determiner **906** may check whether a representation of the UI element is cached in memory **316** based on determining that a UI element is relatively complex.

[0151] FIG. **13** is a flowchart for a method **1300** for executing a set of drawings instructions when a representation of a UI element corresponding to the set of drawing instructions is not cached, according to some embodiments. Method **1300** can be performed by processing logic that can comprise hardware (e.g., circuitry, dedicated logic, programmable logic, microcode, etc.), software (e.g., instructions executing on a processing device), or a combination thereof. It is to be appreciated that not all steps may be needed to perform the disclosure provided herein. Further, some of the steps may be performed simultaneously, or in a different order than shown in FIG. **13**, as will be understood by a person of ordinary skill in the art.

[0152] Method **1300** shall be described with reference to media device **900** of FIG. **9**. However, method **1300** is not limited to that example embodiment.

[0153] In **1302**, GUI renderer **904** may obtain a first set of drawing instructions configured to render a second UI element for a GUI.

[0154] In **1304**, GUI renderer **904** may determine that a representation of the second UI element is not stored in memory **916**. Additional details regarding determining that the representation of the second UI element is not stored in memory **916** are provided below with reference to FIG. **14**.

[0155] Operations **1306**, **1308**, and **1308** may be performed in response to determining that the representation of the second UI element is not stored in memory **916**. For instance, in **1306**, GUI renderer **904** may execute the second set of drawing instructions. In **1308**, GUI renderer **904** may store the representation of the second UI element from memory **916**. In **1310**, GUI renderer **904** may render the obtained representation of the second UI element for the GUI. For example, GUI renderer **904** may render the representation of the second UI element for the GUI.

[0156] FIG. **14** is a flowchart for a method **1400** for determining whether a representation of a second UI element is stored in memory, according to some embodiments. Method **1400** can be performed by processing logic that can comprise hardware (e.g., circuitry, dedicated logic, programmable logic, microcode, etc.), software (e.g., instructions executing on a processing device), or a combination thereof. It is to be appreciated that not all steps may be needed to perform the disclosure provided herein. Further, some of the steps may be performed simultaneously, or in a different order than shown in FIG. **14**, as will be understood by a person of ordinary skill in the art.

[0157] Method **1400** shall be described with reference to media device **900** of FIG. **9**. However, method **1100** is not limited to that example embodiment.

[0158] In **1402**, hash engine **910** may generate a hash value based on the second set of drawing instructions. For example, the second set of drawing instructions may be provided as an input to a hash function of hash engine **910**, which outputs the hash value based on the second set of drawing instructions.

[0159] In **1404**, hash engine **910** may compare the hash value to a set of hash values **920** stored in memory **916**, wherein each hash value of the set of hash values **920** corresponds to a respective set of drawing instructions for rendering a particular user interface element.

[0160] In **1406**, hash engine **910** may determine that the hash value does not match a hash value from the set of hash values **920**.

[0161] In **1408**, hash engine **910** may determine that the representation of the second user interface element is not stored in memory **916** based on determining the hash value does not match any hash

values from the set of hash values **920**.

[0162] FIG. **15** is a flowchart for a method **1500** for rendering a representation of a second UI element, according to some embodiments. Method **1500** can be performed by processing logic that can comprise hardware (e.g., circuitry, dedicated logic, programmable logic, microcode, etc.), software (e.g., instructions executing on a processing device), or a combination thereof. It is to be appreciated that not all steps may be needed to perform the disclosure provided herein. Further, some of the steps may be performed simultaneously, or in a different order than shown in FIG. **15**, as will be understood by a person of ordinary skill in the art.

[0163] Method **1500** shall be described with reference to media device **900** of FIG. **9**. However, method **1500** is not limited to that example embodiment.

[0164] In **1502**, GUI renderer **904** may obtain a second set of drawing instructions configured to render a second UI element for the GUI.

[0165] In **1504**, complexity determiner **906** may determine that a number of drawing instructions in the second set of drawing instructions does not exceed a predetermined threshold.

[0166] In **1506**, GUI renderer **904** may execute the second set of drawing instructions to generate a representation of the second UI element based on determining that the number of drawing instructions in the second set of drawing instructions does not exceed a predetermined threshold. As described above, representations of UI elements that are determined to be relatively simply are not cached in memory **916**, and thus, GUI renderer **904** may generate representations of such UI elements without caching the generated the representations or checking whether such representations are cached.

[0167] In **1508**, GUI renderer **904** may render the representation of the second UI element for the GUI.

Example Computer System

[0168] Various embodiments may be implemented, for example, using one or more well-known computer systems, such as computer system **1600** shown in FIG. **16**. For example, media device **106**, media device **108**, media device **301**, and media device **900** may be implemented using combinations or sub-combinations of computer system **900** to perform various functions described herein, e.g., the operations described in sequence diagram **700**, sequence diagram **800**, method **1000**, method **1100**, method **1200**, method **1300**, method **1400**, and method **1500**. Also or alternatively, one or more computer systems **1600** may be used, for example, to implement any of the embodiments discussed herein, as well as combinations and sub-combinations thereof.

[0169] Computer system **1600** may include one or more processors (also called central processing units, or CPUs), such as a processor **1604**. Processor **1604** may be connected to a communication infrastructure or bus **1606**.

[0170] Computer system **1600** may also include user input/output device(s) **1603**, such as monitors, keyboards, pointing devices, etc., which may communicate with communication infrastructure **1606** through user input/output interface(s) **1602**.

[0171] One or more of processors **1604** may be a graphics processing unit (GPU). In an embodiment, a GPU may be a processor that is a specialized electronic circuit designed to process mathematically intensive applications. The GPU may have a parallel structure that is efficient for parallel processing of large blocks of data, such as mathematically intensive data common to computer graphics applications, images, videos, etc.

[0172] Computer system **1600** may also include a main or primary memory **1608**, such as random access memory (RAM). Main memory **1608** may include one or more levels of cache. Main memory **1608** may have stored therein control logic (i.e., computer software) and/or data.

[0173] Computer system **1600** may also include one or more secondary storage devices or memory **1610**. Secondary memory **1610** may include, for example, a hard disk drive **1612** and/or a removable storage device or drive **1614**. Removable storage drive **1614** may be a floppy disk drive, a magnetic tape drive, a compact disk drive, an optical storage device, tape backup device, and/or

any other storage device/drive.

[0174] Removable storage drive **1614** may interact with a removable storage unit **1618**. Removable storage unit **1618** may include a computer usable or readable storage device having stored thereon computer software (control logic) and/or data. Removable storage unit **1618** may be a floppy disk, magnetic tape, compact disk, DVD, optical storage disk, and/or any other computer data storage device. Removable storage drive **1614** may read from and/or write to removable storage unit **1618**.

[0175] Secondary memory **1610** may include other means, devices, components, instrumentalities or other approaches for allowing computer programs and/or other instructions and/or data to be accessed by computer system **1600**. Such means, devices, components, instrumentalities or other approaches may include, for example, a removable storage unit **1622** and an interface **1620**. Examples of the removable storage unit **1622** and the interface **1620** may include a program cartridge and cartridge interface (such as that found in video game devices), a removable memory chip (such as an EPROM or PROM) and associated socket, a memory stick and USB or other port, a memory card and associated memory card slot, and/or any other removable storage unit and associated interface.

[0176] Computer system **1600** may further include a communication or network interface **1624**. Communication interface **1624** may enable computer system **1600** to communicate and interact with any combination of external devices, external networks, external entities, etc. (individually and collectively referenced by reference number **1628**). For example, communication interface **1624** may allow computer system **1600** to communicate with external or remote devices **1628** over communications path **1626**, which may be wired and/or wireless (or a combination thereof), and which may include any combination of LANs, WANs, the Internet, etc. Control logic and/or data may be transmitted to and from computer system **1600** via communication path **1626**.

[0177] Computer system **1600** may also be any of a personal digital assistant (PDA), desktop workstation, laptop or notebook computer, netbook, tablet, smart phone, smart watch or other wearable, appliance, part of the Internet-of-Things, and/or embedded system, to name a few non-limiting examples, or any combination thereof.

[0178] Computer system **1600** may be a client or server, accessing or hosting any applications and/or data through any delivery paradigm, including but not limited to remote or distributed cloud computing solutions; local or on-premises software (“on-premise” cloud-based solutions); “as a service” models (e.g., content as a service (CaaS), digital content as a service (DCaaS), software as a service (SaaS), managed software as a service (MSaaS), platform as a service (PaaS), desktop as a service (DaaS), framework as a service (FaaS), backend as a service (BaaS), mobile backend as a service (MBaaS), infrastructure as a service (IaaS), etc.); and/or a hybrid model including any combination of the foregoing examples or other services or delivery paradigms.

[0179] Any applicable data structures, file formats, and schemas in computer system **1600** may be derived from standards including but not limited to JavaScript Object Notation (JSON), Extensible Markup Language (XML), Yet Another Markup Language (YAML), Extensible Hypertext Markup Language (XHTML), Wireless Markup Language (WML), MessagePack, XML User Interface Language (XUL), or any other functionally similar representations alone or in combination. Alternatively, proprietary data structures, formats or schemas may be used, either exclusively or in combination with known or open standards.

[0180] In some embodiments, a tangible, non-transitory apparatus or article of manufacture comprising a tangible, non-transitory computer useable or readable medium having control logic (software) stored thereon may also be referred to herein as a computer program product or program storage device. This includes, but is not limited to, computer system **1600**, main memory **1608**, secondary memory **1610**, and removable storage units **1618** and **1622**, as well as tangible articles of manufacture embodying any combination of the foregoing. Such control logic, when executed by one or more data processing devices (such as computer system **1600** or processor(s) **1604**), may cause such data processing devices to operate as described herein.

[0181] Based on the teachings contained in this disclosure, it will be apparent to persons skilled in the relevant art(s) how to make and use embodiments of this disclosure using data processing devices, computer systems and/or computer architectures other than that shown in FIG. 16. In particular, embodiments can operate with software, hardware, and/or operating system implementations other than those described herein.

CONCLUSION

[0182] It is to be appreciated that the Detailed Description section, and not any other section, is intended to be used to interpret the claims. Other sections can set forth one or more but not all exemplary embodiments as contemplated by the inventor(s), and thus, are not intended to limit this disclosure or the appended claims in any way.

[0183] While this disclosure describes exemplary embodiments for exemplary fields and applications, it should be understood that the disclosure is not limited thereto. Other embodiments and modifications thereto are possible, and are within the scope and spirit of this disclosure. For example, and without limiting the generality of this paragraph, embodiments are not limited to the software, hardware, firmware, and/or entities illustrated in the figures and/or described herein. Further, embodiments (whether or not explicitly described herein) have significant utility to fields and applications beyond the examples described herein.

[0184] Embodiments have been described herein with the aid of functional building blocks illustrating the implementation of specified functions and relationships thereof. The boundaries of these functional building blocks have been arbitrarily defined herein for the convenience of the description. Alternate boundaries can be defined as long as the specified functions and relationships (or equivalents thereof) are appropriately performed. Also, alternative embodiments can perform functional blocks, steps, operations, methods, etc. using orderings different than those described herein.

[0185] References herein to “one embodiment,” “an embodiment,” “an example embodiment,” or similar phrases, indicate that the embodiment described may include a particular feature, structure, or characteristic, but every embodiment may not necessarily include the particular feature, structure, or characteristic. Moreover, such phrases are not necessarily referring to the same embodiment. Further, when a particular feature, structure, or characteristic is described in connection with an embodiment, it would be within the knowledge of persons skilled in the relevant art(s) to incorporate such feature, structure, or characteristic into other embodiments whether or not explicitly mentioned or described herein. Additionally, some embodiments can be described using the expression “coupled” and “connected” along with their derivatives. These terms are not necessarily intended as synonyms for each other. For example, some embodiments can be described using the terms “connected” and/or “coupled” to indicate that two or more elements are in direct physical or electrical contact with each other. The term “coupled,” however, can also mean that two or more elements are not in direct contact with each other, but yet still co-operate or interact with each other.

[0186] The breadth and scope of this disclosure should not be limited by any of the above-described exemplary embodiments, but should be defined only in accordance with the following claims and their equivalents.

Claims

1. A computer-implemented method, comprising: obtaining, by at least one computer processor, a first set of drawing instructions configured to render a first user interface element for a graphical user interface; determining that a representation of the first user interface element is stored in a memory; and in response to determining that the representation of the first user interface element is stored in the memory: bypassing execution of the first set of drawing instructions; obtaining the representation of the first user interface element from the memory based on bypassing execution of

the first set of drawing instructions; and rendering the obtained representation of the first user interface element for the graphical user interface.

2. The computer-implemented method of claim 1, wherein determining that the representation of the first user interface element is stored in the memory comprises: generating a hash value based on the first set of drawing instructions; comparing the hash value to a set of hash values stored in the memory, wherein each hash value of the set of hash values corresponds to a respective set of drawing instructions for rendering a particular user interface element; determining the hash value matches a hash value from the set of hash values; and determining that the representation of the first user interface element is stored in the memory based on determining the hash value matches the hash value from the set of hash values.

3. The computer-implemented method of claim 1, wherein determining that the representation of the first user interface element is stored in the memory comprises: determining that a number of drawing instructions in the first set of drawing instructions exceeds a predetermined threshold; and determining that the representation of the first user interface element is stored in the memory based on determining that the number of drawing instructions in the first set of drawing instructions exceeds a predetermined threshold.

4. The computer-implemented method of claim 1, further comprising: obtaining a second set of drawing instructions configured to render a second user interface element for the graphical user interface; determining that a representation of the second user interface element is not stored in the memory; and in response to determining that the representation of the second user interface element is not stored in the memory: executing the second set of drawing instructions to generate a representation of the second user interface element; storing the representation of the second user interface element in the memory; and rendering the representation of the second user interface element for the graphical user interface.

5. The computer-implemented method of claim 4, wherein determining that the representation of the second user interface element is not stored in the memory comprises: generating a hash value based on the second set of drawing instructions; comparing the hash value to a set of hash values stored in the memory, wherein each hash value of the set of hash values corresponds to a respective set of drawing instructions for rendering a particular user interface element; determining the hash value does not match a hash value from the set of hash values; and determining that the representation of the second user interface element is not stored in the memory based on determining the hash value does not match any hash values from the set of hash values.

6. The computer-implemented method of claim 1, further comprising: obtaining a second set of drawing instructions configured to render a second user interface element for the graphical user interface; determining that a number of drawing instructions in the second set of drawing instructions does not exceed a predetermined threshold; executing the second set of drawing instructions to generate a representation of the second user interface element based on determining that the number of drawing instructions in the second set of drawing instructions does not exceed a predetermined threshold; and rendering the representation of the second user interface element for the graphical user interface.

7. The computer-implemented method of claim 1, wherein the representation of the first user interface element comprises a bitmap image representative of the first user interface element.

8. A system, comprising: one or more memories; and at least one processor each coupled to at least one of the one or more memories and configured to perform operations comprising: obtaining a first set of drawing instructions configured to render a first user interface element for a graphical user interface; determining that a representation of the first user interface element is stored in a memory; and in response to determining that the representation of the first user interface element is stored in the memory: bypassing execution of the first set of drawing instructions; obtaining the representation of the first user interface element from the memory based on bypassing execution of the first set of drawing instructions; and rendering the obtained representation of the first user

interface element for the graphical user interface.

9. The system of claim 8, wherein determining that the representation of the first user interface element is stored in the memory comprises: generating a hash value based on the first set of drawing instructions; comparing the hash value to a set of hash values stored in the memory, wherein each hash value of the set of hash values corresponds to a respective set of drawing instructions for rendering a particular user interface element; determining the hash value matches a hash value from the set of hash values; and determining that the representation of the first user interface element is stored in the memory based on determining the hash value matches the hash value from the set of hash values.

10. The system of claim 8, wherein determining that the representation of the first user interface element is stored in the memory comprises: determining that a number of drawing instructions in the first set of drawing instructions exceeds a predetermined threshold; and determining that the representation of the first user interface element is stored in the memory based on determining that the number of drawing instructions in the first set of drawing instructions exceeds a predetermined threshold.

11. The system of claim 8, the operations further comprising: obtaining a second set of drawing instructions configured to render a second user interface element for the graphical user interface; determining that a representation of the second user interface element is not stored in the memory; and in response to determining that the representation of the second user interface element is not stored in the memory: executing the second set of drawing instructions to generate a representation of the second user interface element; storing the representation of the second user interface element in the memory; and rendering the representation of the second user interface element for the graphical user interface.

12. The system of claim 11, wherein determining that the representation of the second user interface element is not stored in the memory comprises: generating a hash value based on the second set of drawing instructions; comparing the hash value to a set of hash values stored in the memory, wherein each hash value of the set of hash values corresponds to a respective set of drawing instructions for rendering a particular user interface element; determining the hash value does not match a hash value from the set of hash values; and determining that the representation of the second user interface element is not stored in the memory based on determining the hash value does not match any hash values from the set of hash values.

13. The system of claim 8, the operations further comprising: obtaining a second set of drawing instructions configured to render a second user interface element for the graphical user interface; determining that a number of drawing instructions in the second set of drawing instructions does not exceed a predetermined threshold; executing the second set of drawing instructions to generate a representation of the second user interface element based on determining that the number of drawing instructions in the second set of drawing instructions does not exceed a predetermined threshold; and rendering the representation of the second user interface element for the graphical user interface.

14. The system of claim 8, wherein the representation of the first user interface element comprises a bitmap image representative of the first user interface element.

15. A non-transitory computer-readable medium having instructions stored thereon that, when executed by at least one computing device, cause the at least one computing device to perform operations comprising: obtaining a first set of drawing instructions configured to render a first user interface element for a graphical user interface; determining that a representation of the first user interface element is stored in a memory; and in response to determining that the representation of the first user interface element is stored in the memory: bypassing execution of the first set of drawing instructions; obtaining the representation of the first user interface element from the memory based on bypassing execution of the first set of drawing instructions; and rendering the obtained representation of the first user interface element for the graphical user interface.

16. The non-transitory computer-readable medium of claim 15, wherein determining that the representation of the first user interface element is stored in the memory comprises: generating a hash value based on the first set of drawing instructions; comparing the hash value to a set of hash values stored in the memory, wherein each hash value of the set of hash values corresponds to a respective set of drawing instructions for rendering a particular user interface element; determining the hash value matches a hash value from the set of hash values; and determining that the representation of the first user interface element is stored in the memory based on determining the hash value matches the hash value from the set of hash values.

17. The non-transitory computer-readable medium of claim 15, wherein determining that the representation of the first user interface element is stored in the memory comprises: determining that a number of drawing instructions in the first set of drawing instructions exceeds a predetermined threshold; and determining that the representation of the first user interface element is stored in the memory based on determining that the number of drawing instructions in the first set of drawing instructions exceeds a predetermined threshold.

18. The non-transitory computer-readable medium of claim 15, the operations further comprising: obtaining a second set of drawing instructions configured to render a second user interface element for the graphical user interface; determining that a representation of the second user interface element is not stored in the memory; and in response to determining that the representation of the second user interface element is not stored in the memory: executing the second set of drawing instructions to generate a representation of the second user interface element; storing the representation of the second user interface element in the memory; and rendering the representation of the second user interface element for the graphical user interface.

19. The non-transitory computer-readable medium of claim 18, wherein determining that the representation of the second user interface element is not stored in the memory comprises: generating a hash value based on the second set of drawing instructions; comparing the hash value to a set of hash values stored in the memory, wherein each hash value of the set of hash values corresponds to a respective set of drawing instructions for rendering a particular user interface element; determining the hash value does not match a hash value from the set of hash values; and determining that the representation of the second user interface element is not stored in the memory based on determining the hash value does not match any hash values from the set of hash values.

20. The non-transitory computer-readable medium of claim 15, further comprising: obtaining a second set of drawing instructions configured to render a second user interface element for the graphical user interface; determining that a number of drawing instructions in the second set of drawing instructions does not exceed a predetermined threshold; executing the second set of drawing instructions to generate a representation of the second user interface element based on determining that the number of drawing instructions in the second set of drawing instructions does not exceed a predetermined threshold; and rendering the representation of the second user interface element for the graphical user interface.
